

AIMLESS – 3-Member Engineering & Integration Specification

SECTION 1 – MEMBER 1

Core / Orchestrator / Infrastructure / Integration

Member 1 owns the system spine, integration lifecycle, and the generation of the final Adobe-compliant report. You are responsible for ensuring AIMLESS executes safely, stays within budget, and produces a valid output even if internal components fail.

1.1 Responsibilities

- **Repository & Packaging:** Owns marketplace.json, Agent Skill validation, and zip packaging (≤50 MB).
- **Entrypoint:** audit-orchestrator is the sole entrypoint.
- **Playwright Host:** Browser lifecycle, context creation, security/network interception, timeouts, and cleanup.
- **Security Gate:** URL validation, SSRF protection, DNS validation, redirect bounding.
- **Budgets:** Request limits (max 6 raw pages), rendering limits (max 3 pages), timeouts (180s target, 270s hard stop).
- **Shared Infrastructure:** Pydantic data models, logging, error handling.
- **Fusion Engine:** Deduplication, cross-signal validation, confidence gating, severity assignment, recommendation selection.
- **Final Report:** JSON schema validation matching Adobe requirements.

1.2 Exact Inputs Member 1 Receives

- **Format:** A single JSON object via CLI or API invocation:
JSON

```
{  
  "input_url": "https://example.com"  
}
```
- **Normalization:** Force https:// by default, lowercase the host, drop URL fragments (#), drop tracking parameters.
- **Rejections:** Block file://, data://, javascript://, localhost, private IP ranges, and explicit credential URLs (user:pass@).

1.3 Exact Outputs Member 1 Produces

- Must emit strict JSON matching the Adobe schema, even upon internal failure (appending issues to coverage):

JSON

```
{
  "site": "https://example.com",
  "audited_at": "2026-09-20T14:32:00Z",
  "audit_version": "1.0.0",
  "summary": {
    "total_findings": 0,
    "critical": 0,
    "high": 0,
    "medium": 0,
    "low": 0,
    "proactive_suggestions": 0
  },
  "coverage": {
    "pages_discovered": 0,
    "pages_sampled_raw": 0,
    "pages_rendered": 0,
    "page_roles_sampled": [],
    "not_observed_roles": [],
    "limitations": [],
    "runtime_ms": 0,
    "external_calls": {"wikidata": 0}
  },
  "findings": [],
  "proactive_suggestions": []
}
```

1.4 Shared Data Models (Pydantic v2)

- **AuditContext (M1 → M2/M3):** target_url (HttpUrl), robots_txt_content (str | None), raw_pages (List[RawPage]), rendered_pages (List[RenderedPage]).
- **RawPage / RenderedPage:** url, status_code, html_content, headers, page_role. RenderedPage adds accessibility_tree (Dict).
- **StructuredFact (M2 → M3/M1):** fact_type (Enum), value (str), source (Enum).
- **CandidateFinding (M2/M3 → M1):** detector_id (str), mechanism (str), confidence (Enum), affected_entity (str), evidence_items (List[Dict]).
- **FinalFinding (M1 → Report):** id, title, severity (Enum), evidence (str), suggested_action (Dict).

1.5 Auditor Interface & Playwright Hand-off (Host/Guest Pattern)

- **M2 (Access-Content) & M3 (Fact-Integrity):** Standard Pydantic-in, Pydantic-out functions. NO side effects or network IO.
- **M3 (Engagement):** Member 1 (Host) initializes the browser, secures it, loads the page, and passes the live Page object to Member 3 (Guest).

Python

M1: src/orchestration/browser_manager.py

```
def run_browser_audit(context: AuditContext, target_url: str) -> List[CandidateFinding]:
```

```
    with sync_playwright() as p:
```

```
        browser = p.chromium.launch() # Run inside the designated sandboxed/containerized
        execution environment
```

```
        page = browser.new_page()
```

```
        setup_security_interception(page)
```

```
        try:
```

```
            page.goto(target_url, timeout=10000, wait_until="domcontentloaded")
```

```
            page.wait_for_timeout(1500) # 1.5s quiet window
```

```
            # M1 passes live page to M3's engagement logic
```

```
            return engagement_auditor.run_interactive_tests(page, context)
```

```
        except Exception as e:
```

```
            context.coverage_notes.append(f"Browser failed: {str(e)}")
```

```
            return []
```

```
        finally:
```

```
            browser.close() # M1 guarantees cleanup
```

1.6 Page Selection

- **Algorithm:** Do not use hardcoded regex alone.
1. Parse homepage links and sitemap links.
 2. Infer roles: landing, detail, editorial, contact, index.
 3. Score by role diversity, same-origin centrality, and canonical uniqueness.
 4. Select max 6 raw pages.
 5. Select max 3 pages for rendering (landing + highest value + fallback).

1.7 Security

- **DNS Resolution:** Use socket.getaddrinfo.
- **IP Blocklist:** Reject IPv4/IPv6 loopback, RFC1918 private ranges, and carrier-grade NAT.
- **Redirects:** Max 5. Validate IP against blocklist *before* following.
- **Playwright Security:** Apply page.route("**/*") to abort media, downloads, websockets, and 3rd-party domains. Block all mutating requests (POST/PUT/DELETE).

1.8 Runtime Budget

- **Raw Fetch:** 10s timeout per page, max 3 concurrent. Max 2MB compressed response cap.
- **Browser Render:** 18s startup max, 10s navigation, 1.5s quiet window. Do NOT wait for network idle.

1.9 Fusion Engine

- **Deduplication:** Merge by (mechanism, normalized affected entity, canonical URL cluster).
- **Cross-Signal Validation:** Elevate severity if render-only issues occur site-wide.
- **Confidence Gate:** Suppress low-confidence claims.
- **Caps:** Max 8 findings (Critical/High first, then max 3 Medium, max 3 Proactive).

1.10 Report Generator

- Map internal CandidateFinding to deterministic Adobe JSON schema.
- Zero LLM usage for findings unless the optional 90-word summary is explicitly enabled.

SECTION 2 – MEMBER 2

Access-Content / AI Discoverability

Member 2 owns public retrieval parsing, structured data validation, and the raw-vs-rendered fact comparison.

2.1 Responsibilities

- **Robots Parsing:** Evaluate AI bot access rules vs. general crawler rules.
- **HTML Parsing:** Extract main, article, headings, and semantic markup.
- **Structured Data:** Parse JSON-LD; validate Organization, Product, Article schemas.
- **Content Extractability:** Identify important facts missing from raw HTML.

2.2 Input Contract

- Receives AuditContext.
- raw_pages are guaranteed.
- rendered_pages and robots_txt_content are optional/nullable.

2.3 Output Contract

- Returns standardized object:

```
JSON
{
  "skill": "access-content-auditor",
  "status": "ok",
  "findings": [ ],
  "coverage": {}
}
```

2.4 Detector Inventory

- **D-01 (Retrieval Access):** High if configured retrieval UA is explicitly denied from primary paths; no finding for training-bot blocks.
- **D-02 (Schema Contradiction):** High for same-entity material conflicts (e.g., mismatched prices). Missing schema is NOT a defect.
- **E-01 (JS Rendering Gap):** Medium/High when core page-role facts appear after render but have no raw/noscript equivalent.
- **E-02 (Non-Text Trap):** Medium if key visual fact lacks machine-readable equivalent (alt/text).

2.5 Robots

- Use urllib.robotparser. Test against a registry (e.g., OAI-SearchBot).
- Evidence: parsed group, normalized path, matching rule.

2.6 Structured Data

- Use extract to parse JSON-LD. Normalize dates to ISO 8601 and prices to floats.
- Explicitly skip schema absence as a defect trigger.

2.7 Important Fact Extraction

- Define StructuredFact keys: organization_name, product_name, price, availability, article_title, contact.
- Search only in main, article, H1, and contact/footer blocks.

2.8 Raw vs Rendered Algorithm

- Extract facts from rendered_html.
- Search for exact/synonym matches in raw_html text, JSON-LD, meta tags, or <noscript>.
- If a core fact >3 tokens (or a price/contact) is unmatched, log as render-only.

2.9 Test Fixtures

- fixture_clean_ssr.html: Price in raw HTML and JSON-LD.
- fixture_js_only.html: Core facts load dynamically via JS.
- fixture_schema_conflict.html: Visible text "\$49" vs JSON-LD "\$59".

SECTION 3 – MEMBER 3

Fact Integrity / Engagement

Member 3 owns first-party factual consistency and bounded accessibility/interaction tests.

PART A – FACT INTEGRITY

Responsibilities & Inputs

- **Input:** AuditContext + StructuredFacts (from M2).
- **Responsibility:** Detect expired claims, explicit internal contradictions, and entity ambiguity.

Wikidata Implementation (Strictly Bounded)

- **Trigger:** Called ONLY if first-party identity is generic, lacks sameAs, and is contradictory.
- **Limits:** Max 1 wbsearchentities request, 3s timeout, max 5 candidates evaluated.
- **Rule:** Multiple external results yield a "Medium risk signal" at worst. Never use external search as absolute truth.

PART B – ENGAGEMENT (HOST/GUEST ARCHITECTURE)

Responsibilities & Inputs

- **Input:** AuditContext plus the capability-limited BrowserAdapter provided by M1 at runtime.
- **Responsibility:** Execute non-mutating interaction traces safely.

Exact Interface (Guest Runner)

```
# M3: src/engagement/engagement_auditor.py
async def run_interactive_tests(
    browser: BrowserAdapter,
    context: AuditContext
) -> List[CandidateFinding]:
    findings = []

    ally_tree = await browser.get_accessibility_tree()

    if detect_blocking_overlay(ally_tree):
        await browser.press("Escape")
        if await browser.is_primary_route_blocked():
            # Try only bounded, explicitly allowed accessible controls.
            await try_safe_accessible_exit(browser, ally_tree)

    if await browser.is_primary_route_blocked():
        focus_state = await bounded_focus_trace(browser, max_steps=8)
        if focus_trapped(focus_state):
            findings.append(
                CandidateFinding(
                    detector_id="G-02",
```

```

        confidence="high",
        # evidence attached here
    )
)

# G-03: infer and test one safe primary route.
# Never submit forms or perform mutating actions.
findings.extend(await test_primary_route(browser, context))

return findings

```

The browser adapter is the only route by which M3 can interact with the page. M3 never accesses the underlying Playwright `Page` directly.

Detectors

- **G-01 (Unnamed Control):** Medium if essential control (menu/submit) has empty computed accessible name.
- **G-02 (Modal Trap):** Critical only if overlay blocks primary route AND Escape/Click/Tab safe exits all fail. Cookie consent is NOT a defect unless it traps focus endlessly.
- **G-03 (Navigation):** Medium if inferred primary route is offered but computationally unreachable or broken.

Test Fixtures

- `fixture_accessible_modal.html`: Dialog closes on Escape.
- `fixture_trap_modal.html`: Overlay blocking clicks with no keydown listeners or close button.
- `fixture_unnamed_control.html`: `<button onclick="submit()"><svg...></button>` with no aria-label.

SHARED CONTRACTS

Object	Producer	Consumer	Format	Required Fields
AuditContext	Member 1	Members 2/3	Pydantic	<code>target_url</code> , <code>raw_pages</code>
BrowserAdapter	Member 1	Member 3	Runtime capability interface	Bounded non-mutating browser operations
StructuredFact	Member 2	Member 3/1	Pydantic	<code>fact_type</code> , <code>value</code> , <code>source</code>
CandidateFinding	Members 2/3	Member 1	Pydantic	<code>detector_id</code> , <code>mechanism</code> , <code>confidence</code>

Object	Producer	Consumer	Format	Required Fields
FinalFinding	Member 1	Report	JSON / Pydantic	id, title, severity, evidence, suggested_action

Versioning

- Schemas reside in src/schemas/v1/.
- Fields added post-freeze must be Optional. Breaking changes require a team sync.

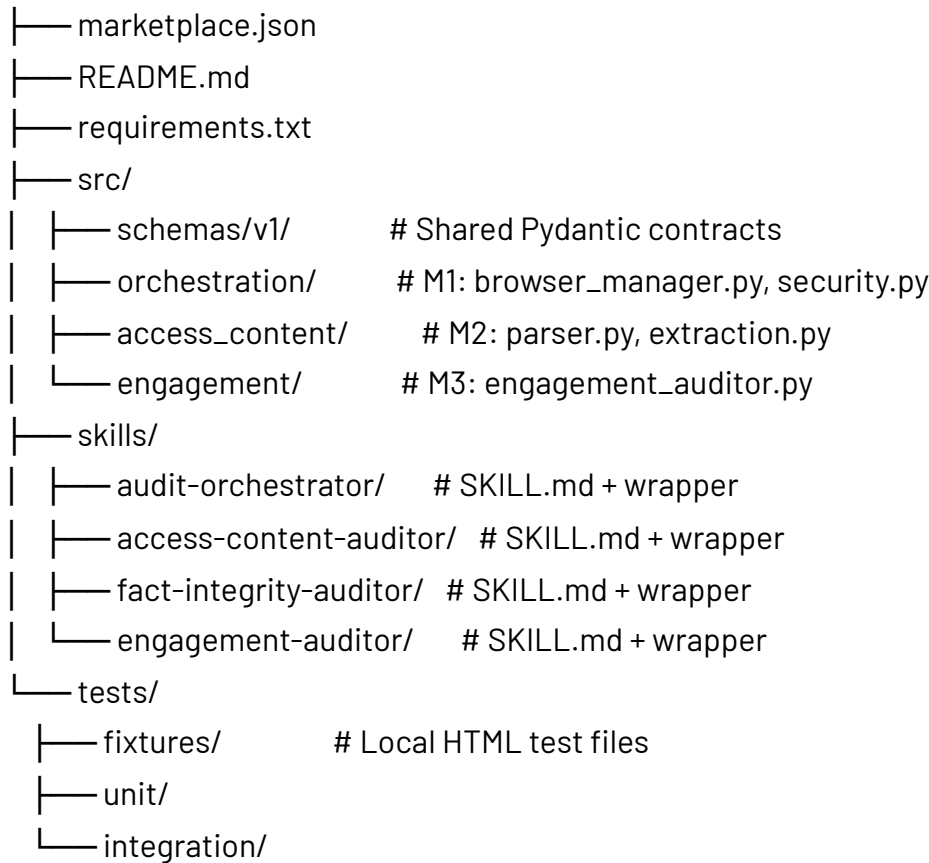
GIT / REPOSITORY WORKFLOW

- **Branches:** main, feature/m1-orchestrator, feature/m2-access-content, feature/m3-fact-engagement.
- **Workflow:**
 1. Schema contract freeze merged to main first.
 2. Parallel development on feature branches.
 3. feature/* branches pull main constantly.
 4. PRs required for main integration.

DIRECTORY STRUCTURE

Plaintext

aimless/



INTEGRATION SEQUENCE

- **Phase 0 – Contract Freeze (Hours 1-4):** All 3 members build src/schemas/v1/. Create marketplace.json and empty SKILL.md files.
- **Phase 1 – Parallel Foundations (Hours 5-20):**
 - M1: Security gate, browser manager, bounded BrowserAdapter, CLI.
 - M2: Robots parser, HTML pipeline, JSON-LD extraction, StructuredFact extraction.
 - M3: Wikidata logic, engagement detector logic against a mocked BrowserAdapter.
- **Phase 2 – Independent Testing (Hours 21-30):** Each member tests logic against local fixtures. M1 mocks M2/M3 payloads.
- **Phase 3 – First Integration (Hours 31-38):** M1 passes AuditContext to M2/M3 and exposes the bounded BrowserAdapter to M3. Run the full end-to-end trace; M3 engagement must remain independent of M2 completion.
- **Phase 4 – Benchmark & Red Team (Hours 39-44):** Test against real unseen URLs. Monitor 180s target and 270s hard stop.
- **Phase 5 – Final Packaging (Hours 45-48):** Zip generation. Check ≤50MB. Validate schemas.

DEPENDENCY GRAPH & EXACT HANDOFFS

- **M1 → M2:** Hands off AuditContext. M2 assumes the target URL has passed M1's security gate and that supplied snapshots are already bounded.
- **M1 → M3 Fact Integrity:** Hands off AuditContext and any available normalized StructuredFacts. M3 must still run fact-integrity checks with partial coverage if M2 fails.
- **M1 → M3 Engagement:** Hands off a capability-limited BrowserAdapter plus AuditContext. M3 never receives unrestricted Playwright control.
- **M2 → M1:** Hands off StructuredFacts and CandidateFindings. This is not a hard dependency for M3 engagement.
- **M3 → M1:** Hands off CandidateFindings from fact-integrity and engagement checks.
- **M2/M3 → M1:** All findings/evidence must validate against the shared Pydantic v1 contract before entering fusion.

"IF I FINISH FIRST" SECTION

- **Member 1:** Build the CLI JSON output formatter. Harden SSRF unit tests (e.g., DNS rebinding).
- **Member 2:** Build complex edge-case HTML fixtures (e.g., malformed JSON-LD, hidden nodes).
- **Member 3:** Refine the G-02 keyboard trace logic. Build a11y tree fixtures.

FAILURE ISOLATION & INTEGRATION RULES

- An M2 failure must not prevent M3 engagement checks from running.
- An M3 fact-integrity failure must not prevent M3 engagement checks from running.
- A single detector exception must be captured as a coverage limitation and must not terminate the entire audit.
- Browser-host failures are owned by M1; detector failures are owned by the corresponding auditor.
- No auditor may bypass M1 security, budgets, or browser capabilities.
- All cross-member objects are validated at the boundary using the shared Pydantic v1 schemas.
- BrowserAdapter calls are bounded and auditable; the underlying Playwright object never crosses the module boundary.

TESTING STRATEGY

- **Rule:** No detector is written without a corresponding HTML fixture.
- **Clean Controls:** Fixtures simulating perfect behavior must yield **0 serious findings**.
- **Broken Controls:** Each fixture defines an expected finding set. The test passes when expected defects are detected with the specified evidence/severity tolerance and no forbidden serious findings are produced.
- **Security Tests:** Host a local server on 127.0.0.1. Assert M1 strictly rejects auditing it. Assert redirects to private IPs fail safely.

FINAL DEFINITION OF DONE

Member 1 DONE

- ☐ URL gate correctly blocks 127.0.0.1, file://, and malicious redirects.
- ☐ Playwright host enforces security interception and per-operation/global timeouts. \n- [] BrowserAdapter exposes only bounded non-mutating operations to Member 3.
- ☐ Fusion engine deduplicates findings and applies max-8 cap.
- ☐ Report strictly matches Adobe JSON schema.

Member 2 DONE

- ☐ Accurately extracts JSON-LD without crashing on malformed tags.
- ☐ Identifies facts present *only* in rendered HTML.
- ☐ SKILL.md validated.

Member 3 DONE

- ☐ Wikidata logic makes max 1 call, times out at 3s.

- ☐ Modal trace safely presses Escape/Tab and observes focus change through the BrowserAdapter.
- ☐ SKILL.md validated.

FINAL SYSTEM DONE

- ☐ Single zip $\leq$ 50MB; no Chromium binaries included.
- ☐ Runtime < 180s on a typical site, hard kill at 270s.
- ☐ Zero LLM hallucination in findings (deterministic templates only).
- ☐ Exactly one endpoint specified in marketplace.json.

CAN THREE PEOPLE ACTUALLY BUILD THIS IN PARALLEL?

Yes. The decoupled nature of the HTTP phase (M2), Interaction phase (M3), and Orchestration phase (M1) allows for simultaneous coding once the Pydantic schemas and Playwright interface are locked.

- **What can be developed independently?** M2's raw parsing logic and M1's security gate.
- **What must be agreed upon first?** src/schemas/v1/ and the Host/Guest Playwright function signature.
- **What is the critical path?** M1's browser_manager.py. M3 cannot execute live interaction tests until M1 can securely initialize and hand off the Playwright Page object.
- **Biggest Integration Risk:** Unhandled Playwright exceptions in M3's logic crashing M1's host context. Strict try/finally blocks are mandatory.